

System Design Document: FitSlot

Challenge: 2-Day System Design & Build Challenge (MERN + PostgreSQL + Redis)

Author: Manya Patkar

Repository: github.com/manyapatkar974/GYM_SLOT

Date: August 2026

1. Problem Understanding & Assumptions

1.1 Problem Statement

The objective is to design and build a high-concurrency gym slot reservation engine where users can discover scheduled sessions, reserve open slots, and cancel bookings. Each slot has a strictly enforced capacity limit of **10 participants**.

1.2 Core Concurrency Challenge

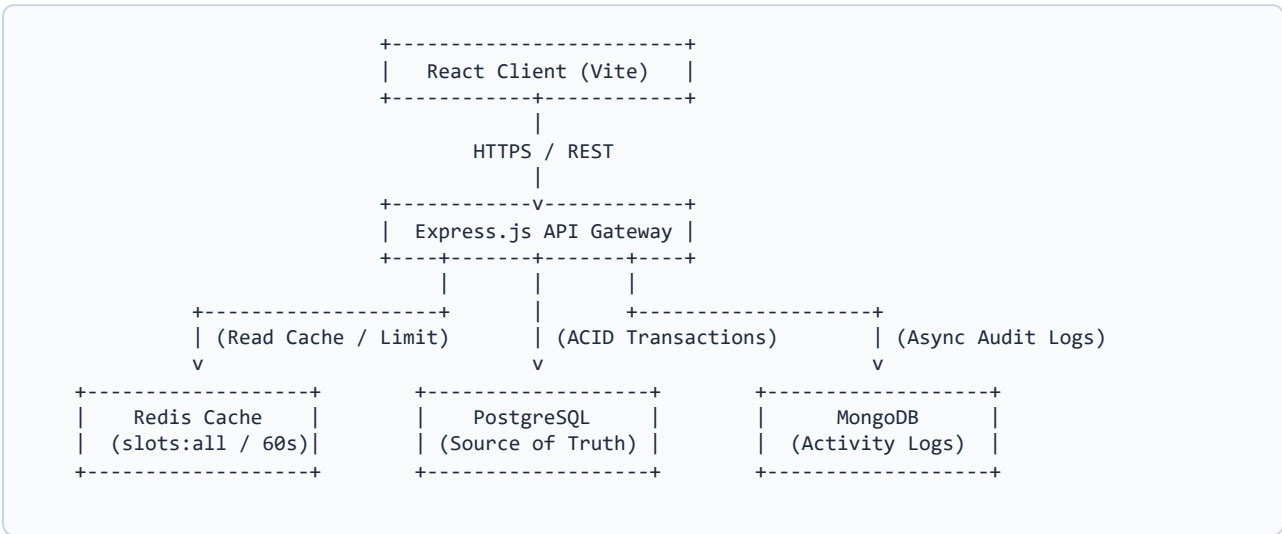
The primary technical hurdle occurs when a slot has only 1 remaining opening (`booked_count = 9` , `capacity = 10`) and multiple users attempt to book at the exact same millisecond.

- In an unmanaged system, concurrent threads simultaneously read `booked_count = 9` , pass the capacity check, insert duplicate bookings, and increment capacity to 11 or higher (Read-Check-Write race condition).
- Our system must guarantee that **exactly one** booking succeeds, all competing requests are cleanly rejected with `HTTP 409 Conflict` , and `booked_count` never exceeds 10 or goes negative.

1.3 Key Assumptions

1. Fixed slot capacity of 10 users per session.
 2. Users cannot hold more than one active booking for the same slot.
 3. Cancellations immediately restore capacity and invalidate caches.
 4. Relational integrity is strictly required for reservation accuracy; audit trails can be eventually consistent.
-

2. High-Level Design (HLD)



Component Breakdown

- **Client Tier (React 18 + Tailwind CSS):** Responsive dashboard with live capacity meters and active booking management.
- **Application Tier (Node.js + Express.js):** Stateless REST API enforcing input validation, JWT authentication, and transactional business logic.
- **Primary Database (PostgreSQL 15):** Relational source of truth handling Users, Slots, and Bookings via ACID transactions and row-level locks.
- **Cache & Rate Limiter (Redis 7):** Ephemeral in-memory store caching hot slot queries (`slots:all`) and applying write rate-limiting.
- **Secondary Database (MongoDB 6):** Non-relational document store for high-throughput, non-blocking user activity and audit logging.

3. Database Architecture & Storage Rationale

3.1 PostgreSQL (Relational Source of Truth)

PostgreSQL was selected for core transactional entities because gym reservations require strict ACID guarantees, foreign-key constraints, and pessimistic locking primitives.

Table: `users`

Column	Type	Constraints	Description
<code>id</code>	UUID	PRIMARY KEY, default <code>uuid_generate_v4()</code>	Unique user identifier
<code>name</code>	VARCHAR(255)	NOT NULL	Full name of the user
<code>email</code>	VARCHAR(255)	UNIQUE, NOT NULL	Unique login email address
<code>password_hash</code>	VARCHAR(255)	NOT NULL	<code>bcrypt</code> hashed password
<code>created_at</code>	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Account creation timestamp
<code>updated_at</code>	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Account update timestamp

Table: `slots`

Column	Type	Constraints	Description
<code>id</code>	UUID	PRIMARY KEY, default <code>uuid_generate_v4()</code>	Unique slot identifier
<code>date</code>	DATE	NOT NULL	Scheduled slot date
<code>start_time</code>	TIME	NOT NULL	Slot start time (e.g. 06:00:00)
<code>end_time</code>	TIME	NOT NULL	Slot end time (e.g. 07:00:00)
<code>capacity</code>	INT	DEFAULT 10	Maximum attendees allowed (10)
<code>booked_count</code>	INT	DEFAULT 0	Current active attendees
<code>created_at</code>	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Creation timestamp
<code>updated_at</code>	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Last updated timestamp

Table: `bookings`

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY, default <code>uuid_generate_v4()</code>	Unique booking identifier
user_id	UUID	FOREIGN KEY (<code>users.id</code>) ON DELETE CASCADE	Reserving user
slot_id	UUID	FOREIGN KEY (<code>slots.id</code>) ON DELETE CASCADE	Target slot
status	ENUM	<code>'ACTIVE'</code> , <code>'CANCELLED'</code>	Current booking state
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Reservation timestamp
cancelled_at	TIMESTAMP	NULL	Cancellation timestamp
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Modification timestamp

3.2 MongoDB (Activity & Audit Collection)

```
{
  "_id": ObjectId("65d8f1e29c8e1a0012ab34cd"),
  "userId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "action": "BOOK_SLOT",
  "slotId": "8a32b384-2191-4df2-bc6d-74d1a29367c1",
  "bookingId": "c71120f2-e567-4f11-9a76-0bf17cb3d119",
  "timestamp": ISODate("2026-08-26T10:00:00.000Z"),
  "metadata": {}
}
```

4. Concurrency Strategy & Row-Level Locking

To serialize booking attempts without application-level bottlenecks, FitSlot uses PostgreSQL's row-level lock within an explicit transaction.

```
BEGIN;

-- 1. Acquire exclusive row-level lock on the target slot row
SELECT id, capacity, booked_count
FROM slots
WHERE id = $1
FOR UPDATE;

-- 2. Verify capacity in business logic:
-- If (booked_count >= capacity) => ROLLBACK & throw HTTP 409 Conflict

-- 3. Insert Active Booking Record (Protected by partial unique index)
INSERT INTO bookings (user_id, slot_id, status)
VALUES ($2, $1, 'ACTIVE');

-- 4. Atomically increment slot booked count
UPDATE slots
SET booked_count = booked_count + 1, updated_at = CURRENT_TIMESTAMP
WHERE id = $1;

-- 5. Commit transaction and release row lock
COMMIT;
```

Unique Constraint for Duplicate Active Bookings

```
CREATE UNIQUE INDEX unique_active_booking
ON bookings (user_id, slot_id)
WHERE status = 'ACTIVE';
```

5. Redis Caching & Rate Limiting Strategy

- **Cache Key:** `slots:all` (all slots) and `slots:{slotId}` (individual slot).
 - **TTL:** 60 seconds expiry to prevent stale drift.
 - **Write Invalidation:** Successful bookings and cancellations execute `DEL slots:all slots:{slotId}` immediately upon transaction commit.
 - **Fail-Safe Fallback:** If Redis is offline, queries cleanly fall back to PostgreSQL with zero downtime.
 - **Rate Limiting:** Enforced on `POST /api/bookings` at 15 requests/minute per IP.
-

6. REST API Contract

Method	Endpoint	Auth	Request Body	Success Response	Error Codes
POST	/api/auth/register	No	{ name, email, password }	201 Created + Token	400, 409
POST	/api/auth/login	No	{ email, password }	200 OK + Token	400, 401
GET	/api/slots	No	None	200 OK + Slots Array	500
GET	/api/slots/:id	No	None	200 OK + Slot Object	404
POST	/api/bookings	Yes	{ slotId }	201 Created + Booking	400, 401, 404, 409, 429
GET	/api/bookings/my	Yes	None	200 OK + Bookings Array	401
DELETE	/api/bookings/:id	Yes	None	200 OK + Cancel Status	400, 401, 404
GET	/api/health	No	None	200 OK + Health Status	503

7. Scalability to 100x Traffic

1. **Stateless API Clustering:** Horizontal scaling of Node.js instances behind an NGINX / AWS Application Load Balancer.
2. **PostgreSQL Read Replicas:** Direct all GET /api/slots reads to read replicas, preserving the primary master for write transactions.
3. **PgBouncer Connection Pooling:** Pool thousands of client connections across backend replicas.
4. **Distributed Redis Cluster:** Shard Redis across multi-AZ clusters for high-availability caching.

8. Concurrency Test Results

=====

⚡ STARTING FITSLOT CONCURRENCY & ROW-LOCKING VERIFICATION TEST ⚡

=====

📌 Test Slot Initialized:

- Total Capacity: 10 | Initial Bookings: 9 | Spots Available: 1

🚀 Dispatching 3 concurrent booking requests via Promise.all()...

📊 Concurrent Request Execution Results:

(index)	user	status	reason	durationMs
0	'Athlete One'	'SUCCESS'	'BOOKED_CONFIRMED (201)'	12
1	'Athlete Two'	'FAILED'	'SLOT_FULL (409 Conflict)'	15
2	'Athlete Three'	'FAILED'	'SLOT_FULL (409 Conflict)'	16

🔍 Verification Assertions:

- Expected Successes: 1 | Actual: 1
- Expected Rejections: 2 | Actual: 2
- Final Slot Booked Count: 10 / 10

✅ TEST PASSED: Zero overbooking detected. Row-level locks verified!

=====